Functions in C

Sudeep K S, Mahindra University (October 2026)

A **function** in C is essentially a piece of code that can be **'reused'**. It does some sequence of operations on the parameters (or arguments) provided. It may or may not return a value.

Syntax:

```
<return type> <function name>(<type of parameter> <parameter name>, …)
{
    <function body>
}
```

For example, below is a function that computes the sum of two integers:

```
int sum(int a, int b)
{
    return (a + b);
}
```

This function **returns a value**: the sum of the two numbers passed to the function.

Both the parameters (a and b) passed to it are integers, the value returned is also an integer.

This can be called from any part of the program, with any two integer values.

Like this:

```
a = 5; b = 7;
c = sum(a, b); // c gets value 12
d = sum(10, 15); // d gets value 25
```

When we have a function, we can call it with different parameters as seen in the example here, and we can avoid repeating the logic of exactly how to calculate sum of two numbers.

Similarly, we can write a function to find the square of a real number: it takes a single argument and returns the square of that number. The body would be `return (a * a);`

Note that 'scanf' and 'printf' are functions that we have been calling with different arguments (parameters) in our programs.

Slightly longer (and non-trivial) functions would be

(i) to find the sum 1 + 2 + … + n, when an integer number n is given;
(ii) to check if a given number is prime or not; or
(iii) given two integers a and b, find the GCD of these two numbers. (See code below)

```c
int sum_upto_n(int n)
{
    int sum, i;
    if (n < 1) { // print a message if n is less than 1
       printf("n should be greater than or equal to 1\n");
       return 0;
    }
    sum = 0;
    for (i = 1; i <= n; i++)
       sum = sum + i;
    return sum; // returns 1 + 2 + ... + n when n >= 1
}
// function to check if a given number is prime
bool prime(int n)
{
    int i;
    if (n == 1) return false; // 1 is not a prime number
    if (n == 2) return true; // 2 is a prime number
    for (i = 2; i < n; i++)
    {
       if (n % i == 0) // found a factor of n (other than 1)
          return false; // so not a prime number
    }
    return true;
}

int gcd(int a, int b)
{
    int temp;
    while (b != 0) {
        temp = b;
        b = a % b; // second number now becomes a mod b
        a = temp; // first number is old b
    } // stops when the second number becomes 0
    return a;
}
```

In this function, we assume that a ≥ b. If we are not sure, we can first swap the values if b > a.

```
    // get the higher number as a
    if (a < b) {
        temp = a;
        a = b;
        b = temp;
    } // now we can be sure that a >= b
```

We can pass an array also as an argument to a function. See the examples below.

1. A function that takes an array (of marks) and n, and prints those marks.

```
void printmarks(int marks[], int n)
{
   int i;
   for (i = 0; i < n; i++)
      printf("marks of student %d is %d\n", i+1, marks[i]);
}
```

Run it and check how it prints the marks given in the array.

2. A function that finds the highest element from an integer array of length n.

```
int highest(int marks[], int n)
{
   int i, max;
   max = marks[0];

   for (i = 1; i < n; i++)
   {
       if (marks[i] > max)
           max = marks[i];
   }

   return max;
}
```

Note that we did not specify the length of the array when we give the array name in the function definition. We used just a square bracket instead. This works, and we can call it on arrays of any size. However, if we are passing a 2-dimensional array as an input parameter to a function, it is necessary to specify the second index size (the number of columns).

This can be generalized for arrays of any dimension. Except for the fist, all other sizes need to be specified in the function definition. (You may specify the first index size also, it is optional.)

We will see an example first, then think why this is needed.

```
// print an m x n matrix, where n = 3
void printmatrix(int matrix[][3], int m, int n)
{
   int i, j;
   for (i = 0; i < m; i++)
   {
      for (j = 0; j < n; j++)
         printf("%d ", matrix[i][j]);
      printf("\n"); // adds a line break after each row
   }
}
```

Call this function and see how it works.

Here, what goes into the function as the the machine needs to know the number of columns because the numbers are stored sequentially in the memory of the machine (computer). If it is a 10 x 3 array, the 30 integer values are stored one after the other, in the order matrix[0][0], matrix[0][1], matrix[0][2], matrix[1][0], matrix[1][1], matrix[1][2], … , matrix[9][0], matrix[9][1], matrix[9][2]. See figure below.

matrix[0][0]
matrix[0][1]
matrix[0][2]
matrix[1][0]
…
…
…
matrix[9][2]

If the machine knows each row contains 3 elements (that is, number of columns = 3), it can easily identify the location of matrix[i][j] when i and j are known. For example, when i = 1 and j = 1, it

knows matrix[1][1] is the fifth element in this block. (i = 1 means first 3 positions can be skipped; and j = 2 means it is the second element from that position).

In general, it can skip (i x 3) + j positions and go to the next element, to get element [i][j].